9.2 Tuples in Python

A **tuple** is a data structure used to store multiple values in one variable.

Tuples are similar to lists, but the main difference is:

List -> mutable -> can be changed

Tuple -> immutable -> cannot be changed

Syntax

tuple_name = (value1, value2, value3)

Explanation

1. Tuples are written using parentheses ().
2. Tuple items are separated by commas.
3. Tuples are ordered, so items have index positions.
4. Tuples are immutable, so items cannot be changed after creation.
5. Tuples allow duplicate values.
6. Tuples can store different data types.

Example 1

```
student = ("Rahul", 21, "Python", 85.5)

print(student)
```

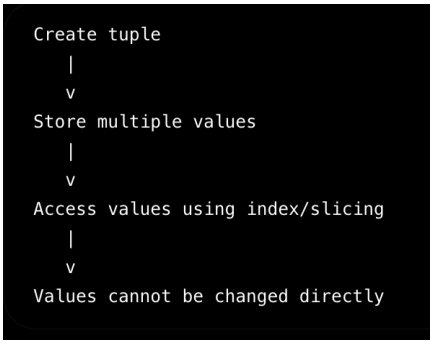
Output: ('Rahul', 21, 'Python', 85.5)

Tuple Properties Table

Property	Meaning	Example
Ordered	Items have fixed positions	items[0]
Immutable	Items cannot be changed	items[1] = 50 not allowed
Allows duplicates	Same value can appear more than once	(10, 10, 20)
Mixed data allowed	Can store different data types	("Aman", 20, True)
Indexed	Every item has position number	0, 1, 2...

CodewithNishchal

Flow Chart



9.2.1 Creating Tuples

Syntax

tuple_name = (item1, item2, item3)

1. Tuples can store numbers, strings, Booleans, or mixed values. An empty tuple can also be created.
2. A tuple can be created with or without parentheses, but parentheses are recommended for clarity.
3. A single-item tuple must have a comma.

Example 1

PYTHON CODE

```
numbers = (10, 20, 30)
names = ("Aman", "Riya", "Kabir")
mixed = ("Python", 100, 99.5, True)
empty_tuple = ()
print(numbers)
print(names)
print(mixed)
print(empty_tuple)
```

Output:

```
(10, 20, 30)
('Aman', 'Riya', 'Kabir')
('Python', 100, 99.5, True)
()
```

Vith Nishchal

Different Ways to Create Tuples

Type	Example
Empty tuple	items = ()
Number tuple	marks = (80, 90, 75)
String tuple	names = ("Aman", "Riya")
Mixed tuple	data = ("Aman", 20, True)
Nested tuple	matrix = ((1, 2), (3, 4))
Without parentheses	items = 10, 20, 30
Using tuple()	letters = tuple("ABC")

9.2.2 Single-Item Tuple

Syntax

tuple_name = (value,)

Explanation

1. A single-item tuple must contain a comma.
2. Without the comma, Python does not treat it as a tuple.
3. Parentheses alone are not enough.

Example 1

PYTHON CODE

```
a = ("Python")
b = ("Python",)
print(type(a))
print(type(b))
```

Output:

```
<class 'str'>
<class 'tuple'>
```

CodeWithNishchal

9.2.3 Accessing Tuple Values

Syntax

tuple_name[index]

1. Tuple values are accessed using index numbers.
2. Python indexing starts from 0.
3. Positive indexing starts from the left.
4. Negative indexing starts from the right.

Index Diagram

```
Tuple:    ("Aman", "Riya", "Kabir", "Neha")
Index:       0       1       2        3
Negative:   -4      -3      -2       -1
```

Example 1

```python
students = ("Aman", "Riya", "Kabir", "Neha")
print(students[0])
print(students[2])
print(students[-1])
```

Output:
Aman
Kabir
Neha

9.2.4 Tuple Slicing

Syntax

tuple_name[start:end:step]

Explanation

1. Slicing is used to get a part of a tuple.
2. The start index is included.
3. The end index is excluded.
4. Step controls the gap between selected values.
5. Slicing returns a new tuple.

CodeWithNishchal

Example 1

```python
numbers = (10, 20, 30, 40, 50, 60)

print(numbers[1:4])
print(numbers[:3])
print(numbers[3:])
print(numbers[::2])
print(numbers[::-1])
```

Output:

(20, 30, 40)
(10, 20, 30)
(40, 50, 60)
(10, 30, 50)
(60, 50, 40, 30, 20, 10)

Slicing Table

Code	Meaning	Result
numbers[1:4]	Index 1 to before 4	(20, 30, 40)
numbers[:3]	Start to before index 3	(10, 20, 30)
numbers[3:]	Index 3 to end	(40, 50, 60)
numbers[::2]	Every second item	(10, 30, 50)
numbers[::-1]	Reverse tuple	(60, 50, 40, 30, 20, 10)

9.2.5 Tuple Immutability

Syntax

tuple_name[index] = new_value

This syntax is **not allowed** for tuples.

Explanation

1. Tuples are immutable.
2. Once a tuple is created, its values cannot be changed directly.
3. We cannot update, add, or remove tuple items directly.
4. If changes are needed, use a list or create a new tuple.

CodeWithNishchal

Example 1

```
PYTHON CODE
numbers = (10, 20, 30)
numbers[1] = 25
```

Output: TypeError: 'tuple' object does not support item assignment

Correct Way

Create a new tuple:

```
PYTHON CODE
numbers = (10, 20, 30)
numbers = (10, 25, 30)
print(numbers)
```

Output: (10, 25, 30)

Memory Idea

Before: numbers ----> (10, 20, 30)
After: numbers ----> (10, 25, 30)
The old tuple is not changed. The variable starts pointing to a new tuple.

9.2.6 Tuple with Mutable Items

Syntax

tuple_name = ([item1, item2], value)

1. A tuple itself is immutable.
2. But if a tuple contains a mutable item like a list, the inner list can be changed.
3. This happens because the tuple is still pointing to the same inner list.

Example 1

```
PYTHON CODE
data = ([10, 20], "Python")
data[0][1] = 99
print(data)
```

Output: ([10, 99], 'Python')

CodeWithNishchal

Explanation Table

Part	Mutable or Immutable?
Outer tuple	Immutable
Inner list	Mutable
data[0] = [1, 2]	Not allowed
data[0][1] = 99	Allowed

9.2.7 Tuple Methods

Method	Purpose	Example	Result
count()	Counts how many times a value appears	items.count(10)	Count of 10
index()	Returns index of first matching value	items.index(20)	Position of 20

Useful Functions with Tuples

Function	Purpose	Example
len()	Counts items	len(numbers)
sum()	Adds numeric items	sum(numbers)
min()	Smallest value	min(numbers)
max()	Largest value	max(numbers)
sorted()	Returns sorted list	sorted(numbers)

9.2.8 Tuple Operations

Operation	Code	Meaning
Concatenation	tuple1 + tuple2	Joins tuples
Repetition	tuple1 * 3	Repeats tuple
Membership	value in tuple1	Checks value exists
Length	len(tuple1)	Counts items
Slicing	tuple1[1:4]	Gets part of tuple

CodeWithNishchal

9.2.9 Tuple Packing

Syntax

tuple_name = value1, value2, value3

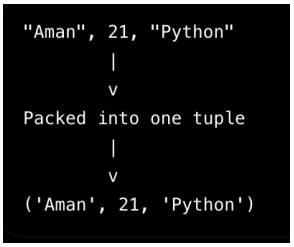
Explanation

1. Tuple packing means storing multiple values together in a tuple.
2. Python automatically packs comma-separated values into a tuple.
3. Parentheses are optional, but recommended for readability.

Example 1

```
PYTHON CODE
student = "Aman", 21, "Python"
print(student)
print(type(student))
```

Output: ('Aman', 21, 'Python')
<class 'tuple'>



9.2.10 Tuple Unpacking

Syntax

var1, var2, var3 = tuple_name

Explanation

1. Tuple unpacking means taking values from a tuple and storing them in separate variables.
2. The number of variables should match the number of tuple values.
3. Unpacking makes code cleaner and readable.

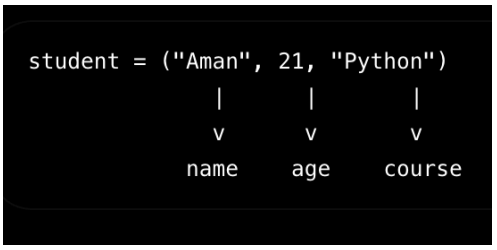
CodeWithNishchal

Example 1

```
student = ("Aman", 21, "Python")

name, age, course = student

print(name)
print(age)
print(course)
```



Output:
Aman
21
Python

Important Point

This gives an error:

```
PYTHON CODE
student = ("Aman", 21, "Python")
name, age = student
```

Output: ValueError: too many values to unpack

Because the tuple has 3 values, but only 2 variables are given.

9.2.11 Extended Tuple Unpacking

Syntax

first, *middle, last = tuple_name

Explanation

1. Extended unpacking is used when we do not want to manually create variables for every value.
2. The starred variable collects extra values.
3. The starred variable becomes a list.

CodeWithNishchal

Example 1

PYTHON CODE

```
numbers = (10, 20, 30, 40, 50)
first, *middle, last = numbers
print(first)
print(middle)
print(last)
```

Output:
10
[20, 30, 40]
50

Important point: The starred variable stores values in a list, not a tuple.

9.2.12 Named Tuples

Syntax

from collections import namedtuple

TupleName = namedtuple("TupleName", ["field1", "field2"])

object_name = TupleName(value1, value2)

Explanation

1. A named tuple is a tuple with named fields.
2. Normal tuple values are accessed by index.
3. Named tuple values can be accessed by name.
4. This makes code more readable.
5. Named tuples are immutable like normal tuples.

Example 1

```
from collections import namedtuple

Student = namedtuple("Student", ["name", "age", "course"])

student1 = Student("Aman", 21, "Python")

print(student1.name)
print(student1.age)
print(student1.course)
```

Output: Aman
21
Python

CodeWithNishchal

Normal Tuple vs Named Tuple

Normal Tuple	Named Tuple
Access by index	Access by name
student[0]	student.name
Less readable	More readable
Immutable	Immutable

9.2.13 Tuple vs List

Point	List	Tuple
Brackets	Uses []	Uses ()
Mutability	Mutable	Immutable
Can update items	Yes	No
Can add/remove items	Yes	No
Speed	Slightly slower	Slightly faster
Best for	Data that may change	Data that should not change

Important Tuple Operations Table

Operation	Code	Meaning
Create tuple	items = (10, 20, 30)	Creates tuple
Single-item tuple	item = (10,)	Creates tuple with one item
Access item	items[0]	Gets first item
Slice tuple	items[1:3]	Gets part of tuple
Count item	items.count(10)	Counts value
Find index	items.index(20)	Finds position
Pack tuple	data = "Aman", 21	Packs values
Unpack tuple	name, age = data	Extracts values
Concatenate	a + b	Joins tuples
Repeat	a * 2	Repeats tuple
Check membership	10 in items	Returns True or False
Convert list to tuple	tuple(my_list)	Creates tuple
Convert tuple to list	list(my_tuple)	Creates list

CodeWithNishchal